Summarize

Chapter 8. Reuse Patterns

Wednesday, February 2, 15:15



As the development team members worked on breaking apart the domain services, they started running into disagreements about what to do with all the shared code and shared functionality. Taylen, upset with what Skyler was doing with regard to the shared code, walked over to Skyler's desk.

"What in the world are you doing?" asked Taylen.

"I'm moving all of the shared code to a new workspace so we can create a shared DLL from it," replied Skyler.

"A *single* shared DLL?"

"That's what I was planning," said Skyler. "Most of the services will need this stuff anyway, so I'm going to create a single DLL that all the services can use."

"That's the worst idea I've ever heard," said Taylen. "Everyone knows you should have multiple shared libraries in a distributed architecture!"

"Not in my opinion," said Sydney. "Seems to me it's much easier to manage a single shared library DLL rather than dozens of them."

"Given that I'm the tech lead for this application, I want you to split that functionality into separate shared libraries."

"OK, OK, I suppose I can move the all of the authorization into its own separate DLL if that would make you happy," said Skyler.

"What?" said Taylen. "The authorization code has to be a shared service, you know——*not* in a shared library.""

"No," said Skyler. "That code should be in a shared DLL."

"What's all the shouting about over there?" asked Addison.

"Taylen wants the authorization functionality to be in a shared service. That's just crazy. I think it should go in the common shared DLL," said Skyler.

"No way," said Taylen. "It's got to be in its own separate shared service."

"And," said Skyler, "Taylen is insisting on having multiple shared libraries for the shared functionality rather than a single shared library."

"Tell you what," said Addison. "Let's go over the trade-offs of shared library granularity, and also go over the trade-offs between a shared library and a shared service to see if we can resolve these issues in a more reasonable and thoughtful manner."

Code reuse is a normal part of software development. Common business domain functionality, such as formatters, calculators, validators, and auditing, are typically shared across multiple components, as is common infrastructure functionality, such as security, logging, and metrics gathering. In most monolithic architectures, code reuse is rarely given a second thought—it's a matter of simply importing or auto-injecting shared class files. However, in distributed architectures, as shown in Figure 8-1, things get a bit more complicated, as questions arise about how to deal with shared functionality.

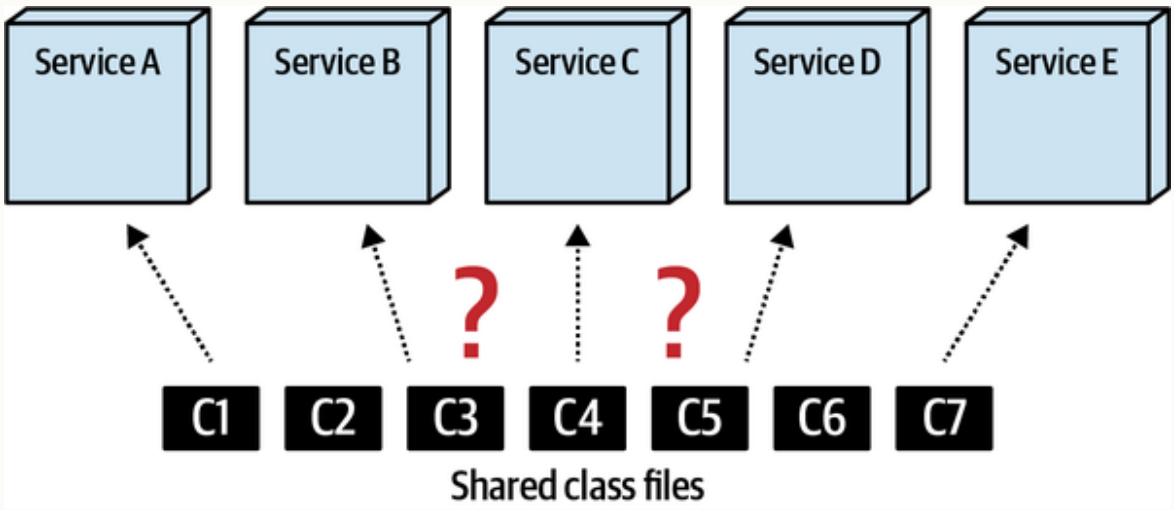


Figure 8-1. Code reuse is a hard part of distributed architecture

Frequently within highly distributed architectures like microservices and serverless environments, phrases like "reuse is abuse!" and "share nothing!" are touted by architects in an attempt to reduce the amount of shared code within these types of architectures. Architects in these environments have even been found to offer countering advice to the famous DRY principle (Don't repeat yourself) by using an opposing acronym called WET (Write every time or Write everything twice).

While developers should try to limit the amount of code reuse within distributed architectures, it is nevertheless a fact of life in software development and must be addressed, particularly in distributed architectures. In this chapter, we introduce several techniques for managing code reuse within a distributed architecture, including replicating code, shared libraries, shared services, and sidecars within a service mesh. For each of these options, we also discuss the pros, cons, and trade-offs of each approach.

Code Replication

In *code replication*, shared code is copied into each service (or more specifically, each service source code repository), as shown in Figure 8-2, thereby avoiding code sharing altogether. While it might sound crazy, this technique became popular in the early days of microservices when a lot of confusion and misunderstanding arose about the *bounded context* concept, hence the drive to create a "share nothing architecture." In theory, code replication seemed like a good approach at that time to reduce code sharing, but in practice it quickly fell apart.

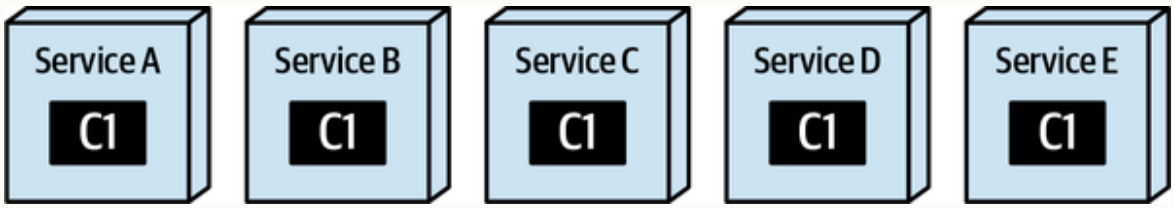


Figure 8-2. With replication, shared functionality is copied into each service

While code replication isn't used much today, it nevertheless is still a valid technique for addressing code reuse across multiple distributed services. This technique should be approached with extreme caution for the obvious reason that if a bug is found in the code or an important change to the code is needed, it would be very difficult and time-consuming to update all services containing the replicated code.

At times, however, this technique can prove useful, particularly for highly static one-off code that most (or all) services need. For example, consider the Java code in Example 8-1 and the corresponding C# code in Example 8-2 that identifies the class in the service that represents the service entry point (usually the restful API class within a service).

Example 8-1. Source code defining a service entry point annotation (Java)

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
public @interface ServiceEntrypoint {}

/* Usage:
```

```
@ServiceEntrypoint
public class PaymentServiceAPI {
  ...
}
*/
```

Example 8-2. Source code defining a service entry point attribute (C#)

```
[AttributeUsage(AttributeTargets.Class)]
class ServiceEntrypoint : Attribute {}

/* Usage:
[ServiceEntrypoint]
class PaymentServiceAPI {
  ...
}
*/
```

Note that the source code in Example 8-1 actually contains no functionality whatsoever. The annotation is simply a marker (or tag) used to identify a particular class as representing the service entry point. However, this simple annotation is very useful for placing other metadata annotations about a particular service, including the service type, domain, bounded context, and so on; see Chapter 89 in *97 Things Every Java Programmer Should Know* by Kevlin Henney and Trisha Gee (O'Reilly) for a description of these metadata custom annotations.

This kind of source code makes a good candidate for replication because it's static and doesn't contain any bugs (and most likely will not in the future). If this were a unique one-off class, it might be worth copying it into each service code repository rather than creating a shared library for it. That said, we generally encourage investigating the other code-sharing techniques presented in this chapter before opting for the code replication technique.

While the replication technique preserves the bounded context, it does make it difficult to apply changes if the code ever does need to be modified. Table 8-1 lists the various trade-offs associated with this technique.

TRADE-OFFS

Table 8-1. Trade-offs for the code replication technique

Advantages	Disadvantages
Preserves the bounded context	Difficult to apply code changes
No code sharing	Code inconsistency across services
	No versioning capabilities across services

When to Use

The replication technique is a good approach when developers have simple static code (like annotations, attributes, simple common utilities, and so on) that is either a one-off class or code that is unlikely to ever change because of defects or functional changes. However, as mentioned earlier, we encourage exploring other code-reuse options before embracing the code replication technique.

When migrating from a monolithic architecture to a distributed one, we've also found that the replication technique can sometimes work for common static utility classes. For example, by replicating a `Utility.cs` C# class to all services, each service can now remove (or enhance) the `Utility.cs` class to suit its particular needs, therefore eliminating unnecessary code and allowing the utility class to evolve for each specific context (similar to the tactical forking technique described in Chapter 3). Again, the risk with this technique is that a defect or change is very difficult to propagate to all services because the code is duplicated for each service.

Shared Library

One of the most common techniques for sharing code is to use a shared library. A shared library is an external artifact (such as a JAR file, DLL, and so on) containing source code that is used by multiple services which is typically bound to the service at compile time (see Figure 8-3). Although the shared library technique seems simple and straightforward, it has its share of complexities and trade-offs, not the least of which is shared library granularity and versioning.

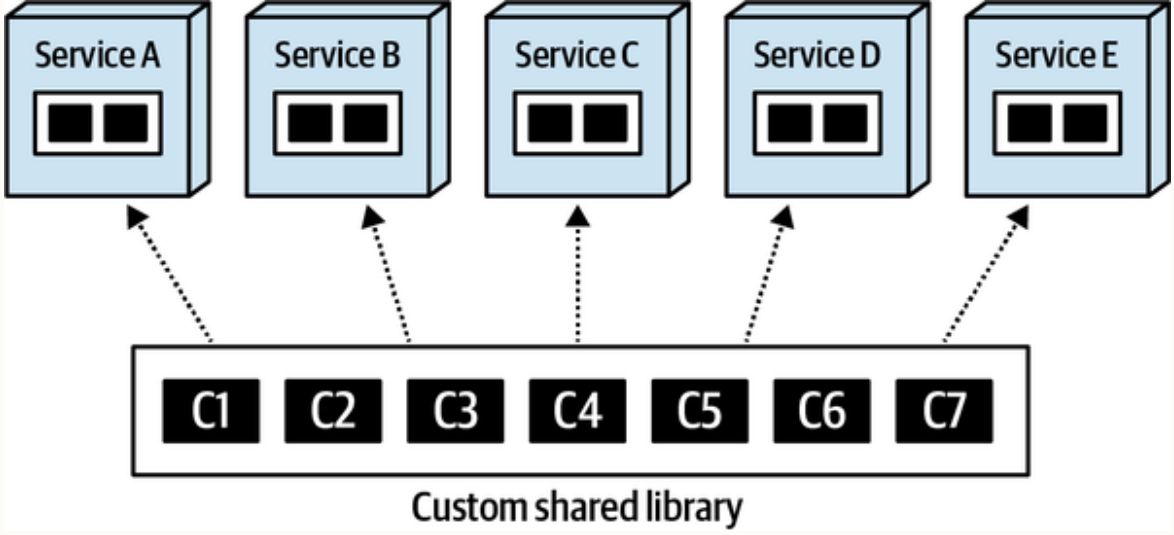


Figure 8-3. With the shared library technique, common code is consolidated and shared at compile time

Dependency Management and Change Control

Similar to service granularity (discussed in Chapter 7), there are trade-offs associated with the granularity of a shared library. The two opposing forces that form trade-offs with shared libraries are dependency management and change control.

Consider the coarse-grained shared library illustrated in Figure 8-4. Note that while the dependency management is relatively straightforward (each service uses the single shared library), change control is not. If a change occurs to any of the class files in the coarse-grained shared library, *every* service, whether it cares about the change or not, must eventually adopt the change because of a version deprecation of the shared library. This forces unnecessary retesting and redeployment of all the services using that library, therefore significantly increasing the overall testing scope of a shared library change.

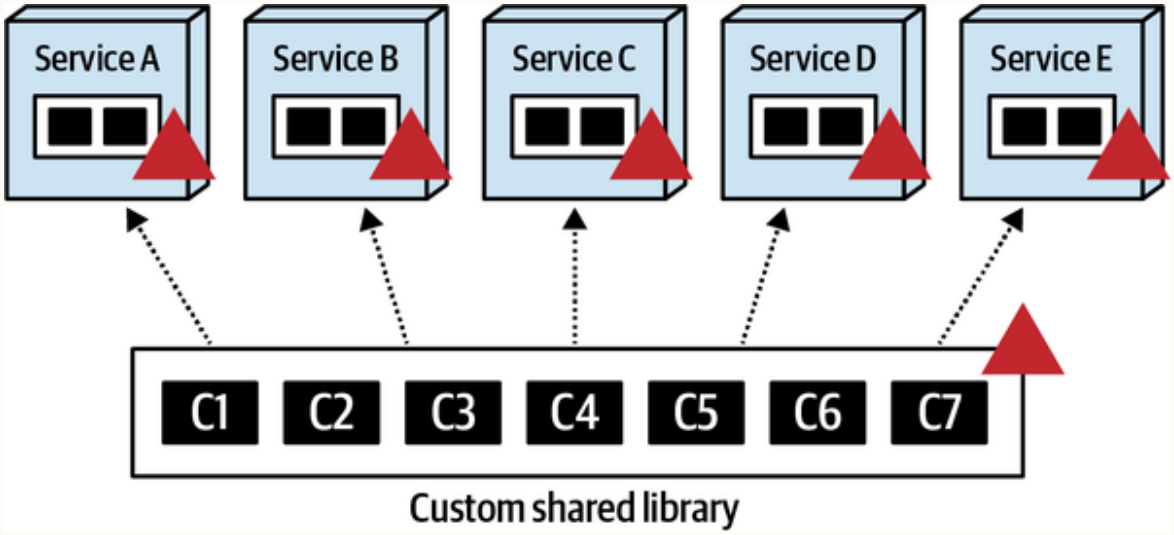


Figure 8-4. Changes to coarse-grained shared libraries impact multiple services but keep dependencies low

Breaking shared code into smaller functionality-based shared libraries (such as security, formatters, annotations, calculators, and so on) is better for change control and overall maintainability, but unfortunately creates a mess in terms of dependency management. As

shown in Figure 8-5, a change in shared class C7 impacts only Service D and Service E, but managing the dependency matrix between shared libraries and services quickly starts looking like a big ball of distributed mud (or what some people refer to as a *distributed monolith*).

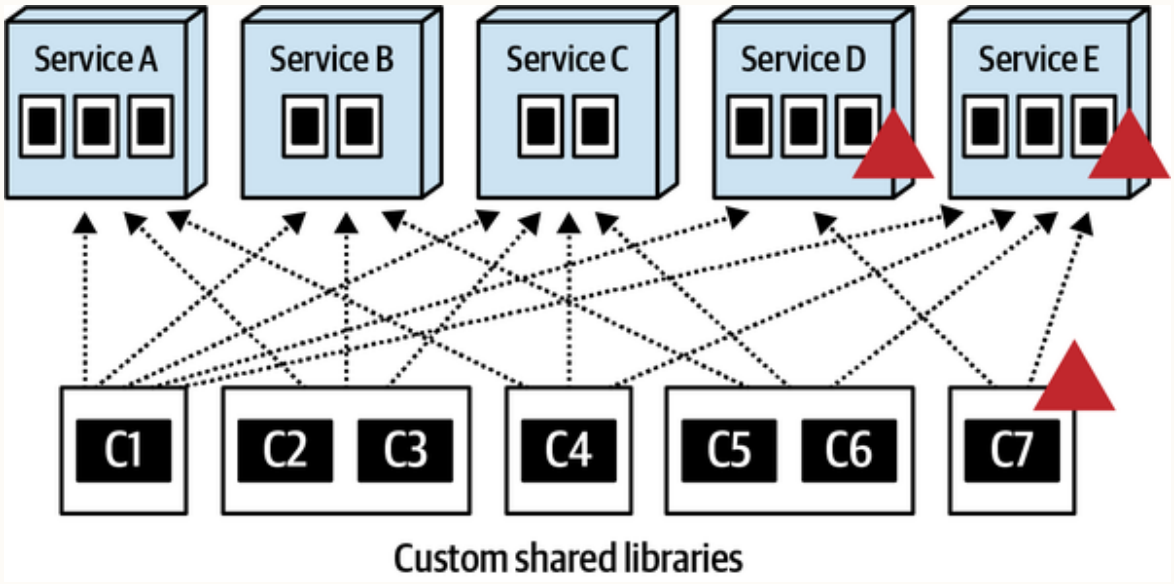


Figure 8-5. Changes to fine-grained shared libraries impact fewer services but increase dependencies

The choice of shared library granularity may not matter much with only a few services, but as the number of services increases, so do the issues associated with change control and dependency management. Just imagine a system with 200 services and 40 shared libraries—it would quickly become overly complex and unmaintainable.

Given these trade-offs of change control and dependency management, our advice is to generally avoid large, coarse-grained shared libraries and strive for smaller, functionally partitioned libraries whenever possible, thus favoring change control over dependency management. For example, carving off relatively static functionality such as formatters and security (authentication and authorization) into their own shared libraries isolates this static code, therefore reducing the testing scope and unnecessary version deprecation deployments for other shared functionality.

Versioning Strategies

Our general advice about shared library versioning is to *always use versioning!* Versioning your shared libraries provides not only backward compatibility, but also a high level of agility—the ability to respond quickly to change.

To illustrate this point, consider a shared library containing common field validation rules called *Validation.jar* that is used by 10 services. Suppose one of those services needs an immediate change to one of the validation rules. By versioning the *Validation.jar* file, the service needing the change can immediately incorporate the new *Validation.jar* version and be

deployed to production right away, without any impact to the other 9 services. Without versioning, all 10 services would have to be tested and redeployed when making the shared library change, thereby increasing the amount of time and coordination for the shared library change (hence less agility).

While the preceding advice may seem obvious, there are trade-offs and hidden complexity in versioning. As a matter of fact, versioning can be so complex that your authors often think of versioning as the ninth fallacy of distributed computing: "versioning is simple"

One of the first complexities of shared library versioning is communicating a version change. In a highly distributed architecture with multiple teams, it is often difficult to communicate a version change to a shared library. How do other teams know that *Validation.jar* just increased to version 1.5? What were the changes? What services are impacted? What teams are impacted? Even with the plethora of tools that manage shared libraries, versions, and change documentation (such as JFrog Artifactory), version changes must nevertheless be coordinated and communicated to the right people at the right time.

Another complexity is the deprecation of older versions of a shared library—removing those versions no longer supported after a certain date. Deprecation strategies range from *custom* (for individual shared libraries) all the way to *global* (for all shared libraries). And, not surprisingly, trade-offs are involved with both approaches.

Assigning a custom deprecation strategy to each shared library is usually the desired approach because libraries change at different rates. For example, if a *Security.jar* shared library doesn't change often, maintaining only two or three versions is a reasonable strategy. However, if the *Calculators.jar* shared library changes weekly, maintaining only two or three versions means that all services using that shared library will be incorporating a newer version on a monthly (or even weekly) basis—causing a lot of unnecessary frequent retesting and redeployment. Therefore, maintaining 10 versions of *Calculators.jar* would be a much more reasonable strategy because of the frequency of change. The trade-off of this approach, however, is that someone must maintain and track the deprecation for *each shared library*. This can sometimes be a daunting task and is definitely not for the faint of heart.

Because change is variable among the various shared libraries, the global deprecation strategy, while simpler, is a less effective approach. The global deprecation strategy dictates that *all* shared libraries, regardless of the rate of change, will not support more than a certain number of backward versions (for example, four). While this is easy to maintain and govern, it can cause significant *churn*--the constant retesting and redeploying of services—just to maintain compatibility with the latest version of a frequently changed shared library. This can drive teams crazy and significantly reduce overall team velocity and productivity.

Regardless of the deprecation strategy used, serious defects or breaking changes to shared code invalidate any sort of deprecation strategy, causing *all* services to adopt the latest version of a shared library at once (or within a very short period of time). This is another reason we recommend keeping shared libraries as fine-grained as appropriate and avoid the coarse-grained *SharedStuff.jar* type of libraries containing all the shared functionality in the system.

One last word of advice regarding versioning: avoid the use of the `LATEST` version when specifying which version of a library a service requires. It has been our experience that services using the `LATEST` version experience issues when doing quick fixes or emergency hot deployments into production, because something in the `LATEST` version might be incompatible with the service, therefore causing additional development and testing effort for the team to release the service into production.

While the shared library technique allows changes to be versioned (therefore providing a good level of agility for shared code changes), dependency management can be difficult and messy. Table 8-2 lists various trade-offs associated with this technique.

TRADE-OFFS

Table 8-2. Trade-offs for the shared library technique

Advantages	Disadvantages
Ability to version changes	Dependencies can be difficult to manage
Shared code is compile-based, reducing runtime errors	Code duplication in heterogeneous codebases
Good agility for code shared code changes	Version deprecation can be difficult
	Version communication can be difficult

When To Use

The shared library technique is a good approach for homogeneous environments where shared code change is low to moderate. The ability to version (although sometimes com-

plex) allows for good levels of agility when making shared code changes. Because shared libraries are usually bound to the service at compile time, operational characteristics such as performance, scalability, and fault tolerance are not impacted, and the risk of breaking other services with a change to common code is low because of versioning.

Shared Service

The primary alternative to using a shared library for common functionality is to use a shared service instead. The *shared service* technique, illustrated in Figure 8-6, avoids reuse by placing shared functionality in a separately deployed service.

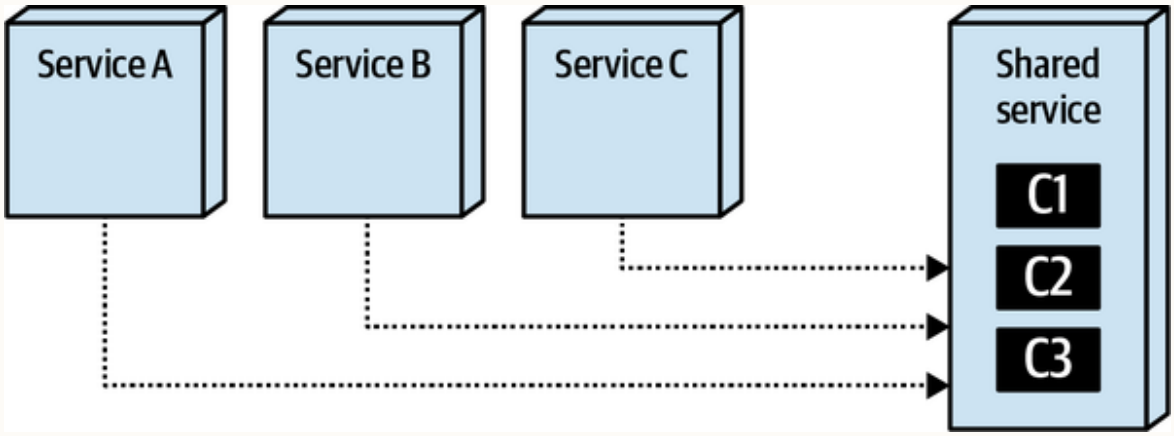


Figure 8-6. With the shared service technique, common functionality is made available at runtime through separate services

One distinguishing factor about the shared service technique is that the shared code must be in the form of *composition*, not *inheritance*. While there is a lot of debate about the use of composition over inheritance from a source code *design* standpoint (see the Thoughtworks article "Composition vs. Inheritance: How to Choose" and Martin Fowler's article "Designed Inheritance"), *architecturally* composition versus inheritance matters when choosing a code-reuse technique, particularly with the shared services technique.

Back in the day, shared services were a common approach to address shared functionality within a distributed architecture. Changes to shared functionality no longer require redeployment of services; rather, since changes are isolated to a separate service, they can be deployed without redeploying other services needing the shared functionality. However, like everything in software architecture, many trade-offs are associated with using shared services, including change risk, performance, scalability, and fault tolerance.

Change Risk

Changing shared functionality using the shared service technique turns out to be a double-edged sword. As illustrated in Figure 8-7, changing shared functionality is simply a matter

of modifying the shared code contained in a separate service (such as a discount calculator), redeploying the service, and voila—the changes are now available to all services, without having to retest and redeploy any other service needing that shared functionality.

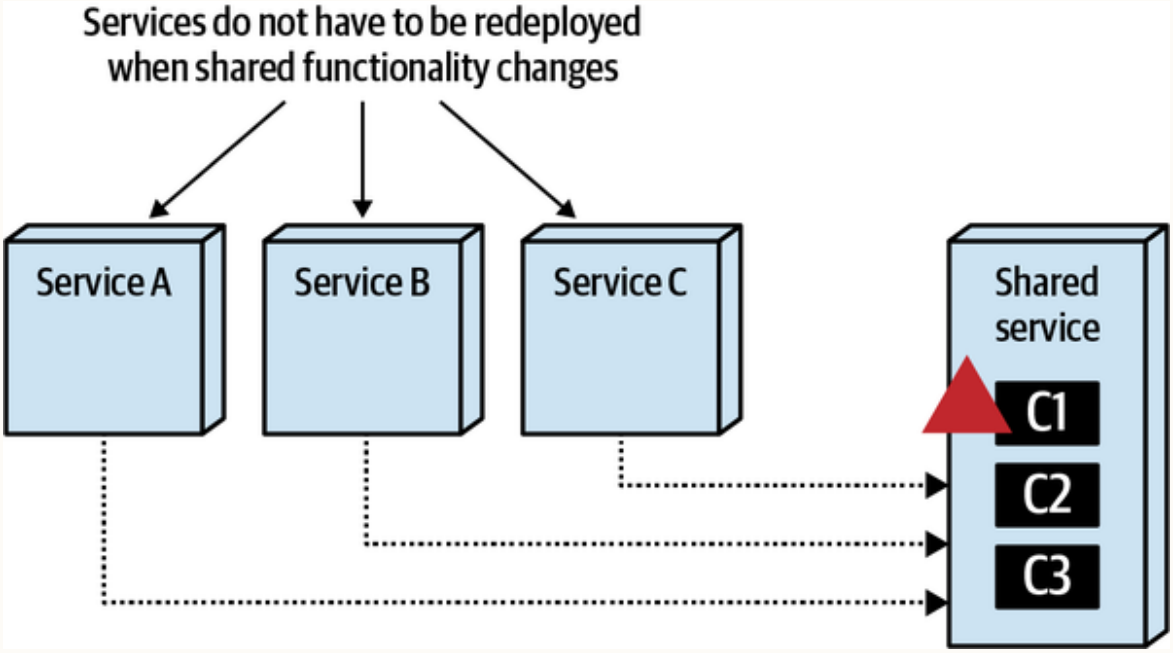


Figure 8-7. Shared functionality changes are isolated to only the shared service

If only life were that simple! The problem, of course, is that a change to a shared service is a *runtime* change, as opposed to a *compile-based* change with the shared library technique. As a result, a "simple" change in a shared service can effectively bring down an entire system, as illustrated in Figure 8-8.

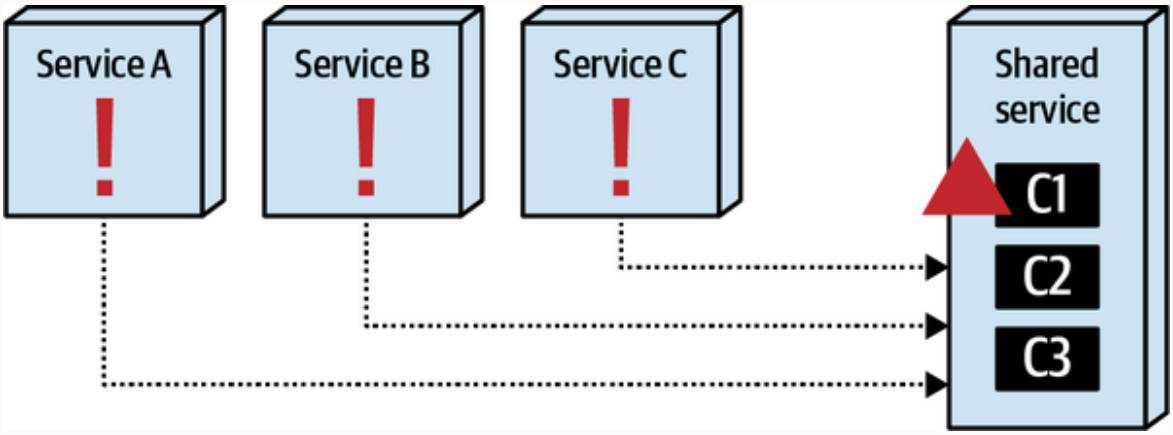


Figure 8-8. Changes to a shared service can break other services at runtime

This necessarily brings to the forefront the topic of versioning. In the shared library technique, versioning is managed through compile-time bindings, significantly reducing risk associated with a change in a shared library. However, how does one version a simple shared service change?

The immediate response, of course, is to use API endpoint versioning—in other words, create a new endpoint containing each shared service change, as shown in Example 8-3.

Example 8-3. Discount calendar with versioning for shared service endpoint

```
app/1.0/discountcalc?orderid=123
app/1.1/discountcalc?orderid=123
app/1.2/discountcalc?orderid=123
app/1.3/discountcalc?orderid=123
latest change -> app/1.4/discountcalc?orderid=123
```

Using this approach, each time a shared service changes, the team would create a new API endpoint containing a new version of the URI. It's not difficult to see the issues that arise with this practice. First of all, services accessing the discount calculator service (or the corresponding configuration for each service) must change to point to the correct version. Second, when should the team create a new API endpoint? What about for a simple error message change? What about for a new calculation? Versioning starts to become largely subjective at this point, *and* the services using the shared service must still change to point to the correct endpoint.

Another problem with API endpoint versioning is that it assumes all access to the shared service is through a RESTful API call going through a gateway or via point-to-point communication. However, in some cases, access to a shared service through interservice communication is commonly done through other types of protocols such as messaging and gRPC (in addition to a RESTful API call). This further complicates the versioning strategy for a change, making it difficult to coordinate versions across multiple protocols.

The bottom line is that with the shared service technique, changes to a shared service are generally runtime in nature, and therefore carry much more risk than with shared libraries. While versioning can help reduce this risk, it's much more complex to apply and manage than that of a shared library.

Performance

Because services requiring the shared functionality must make an interservice call to a shared service, performance is impacted because of network latency (and security latency, assuming the endpoints to the shared service are secure). This trade-off, shown in Figure 8-9, does not exist with the shared library technique when accessing shared code.

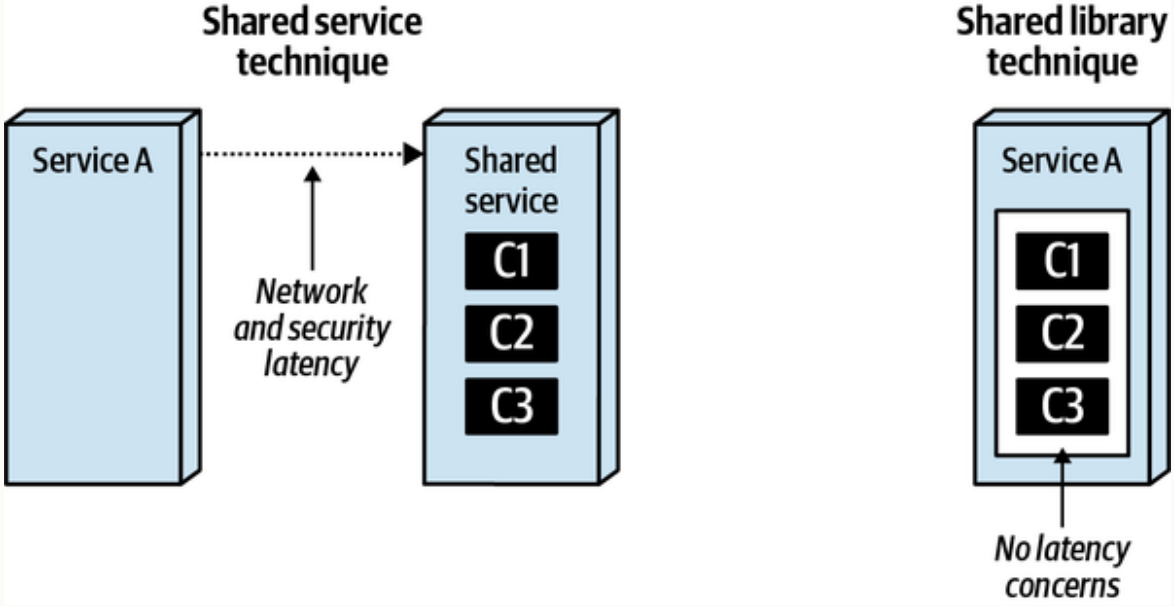


Figure 8-9. Shared service introduces network and security latency

Use of gRPC can help mitigate some of the performance issues by significantly reducing network latency, as can the use of asynchronous protocols like messaging. With messaging, the service needing the shared functionality can issue a request through a request queue, perform other work, and once needed, can retrieve the results through a separate reply queue using a correlation ID (see *Java Message Service*, Second Edition by Mark Richards et al. (O'Reilly) for more information about messaging techniques).

Scalability

Another drawback of the shared service technique is that the shared service must scale as services using the shared service scale. This can sometimes be a mess to manage, particularly with multiple services concurrently accessing the same shared service. However, as illustrated in Figure 8-10, the shared library technique does not have this issue because the shared functionality is contained within the service at compile time.

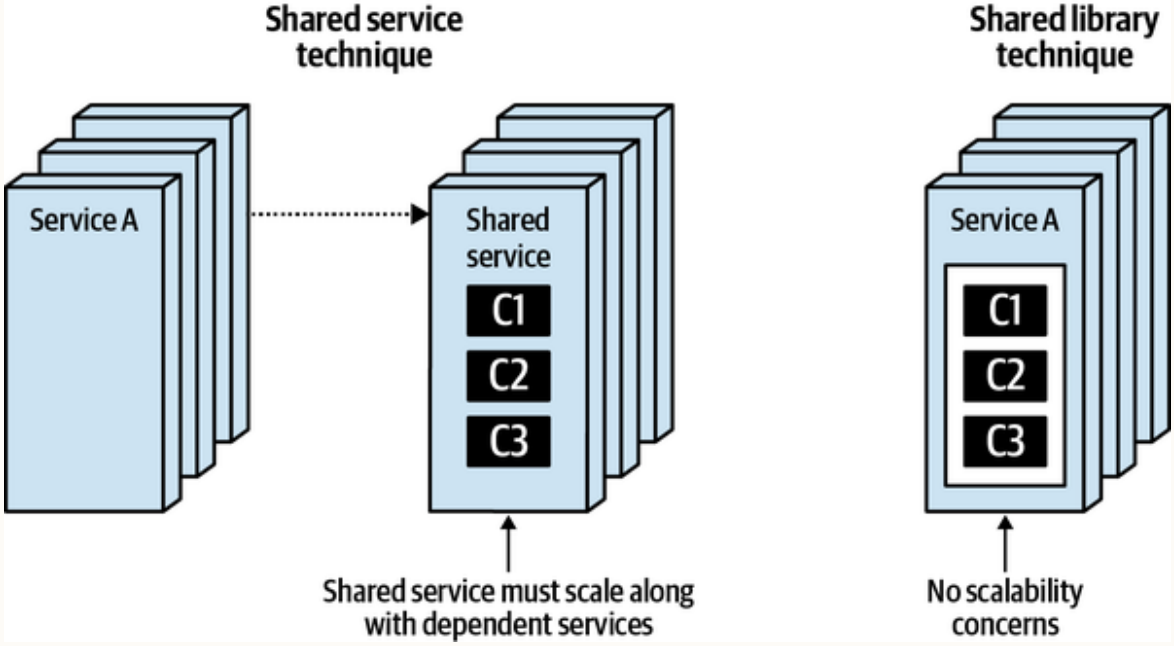


Figure 8-10. Shared services must scale as dependent services scale

Fault Tolerance

While fault-tolerance issues can usually be mitigated through multiple instances of a service, nevertheless it is a trade-off to consider when using the shared service technique. As illustrated in Figure 8-11, if the shared service becomes unavailable, services requiring the shared functionality are rendered nonoperational until the shared service is available. The shared library technique does not have this issue since the shared functionality is contained in the service at compile time, and therefore accessed through standard method or function calls.

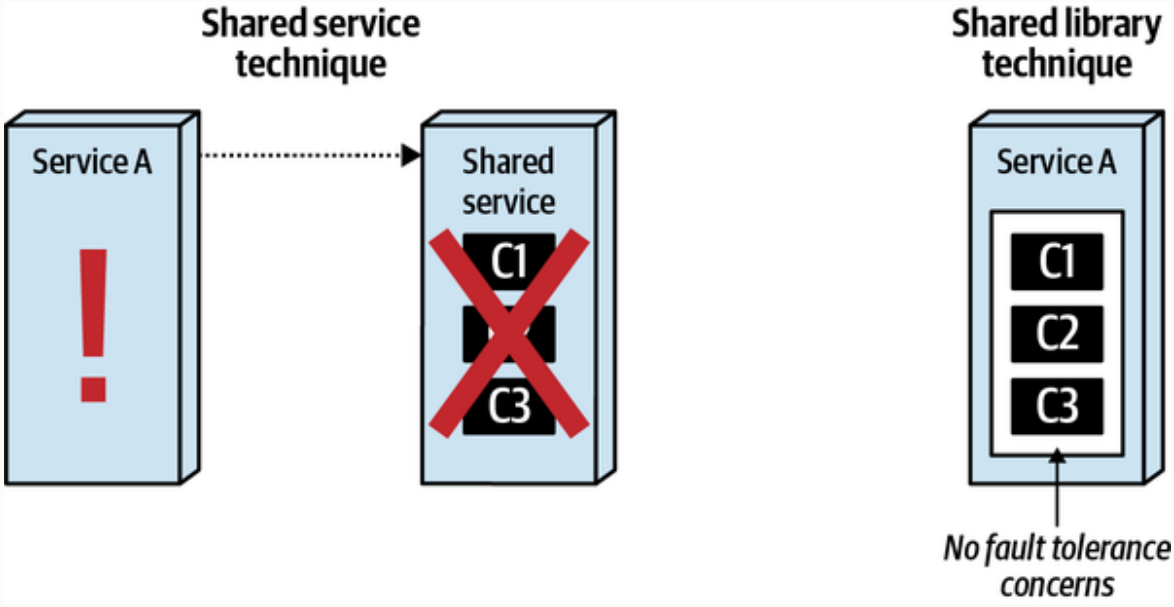


Figure 8-11. Shared services introduce fault-tolerance issues

While the shared service technique preserves the bounded context and is good for shared code that changes frequently, operational characteristics such as performance, scalability, and availability suffer. Table 8-3 lists the various trade-offs associated with this technique.

TRADE-OFFS

Table 8-3. Trade-offs for the shared service technique

Advantages	Disadvantages
Good for high code volatility	Versioning changes can be difficult
No code duplication in heterogeneous codebases	Performance is impacted due to latency
Preserves the bounded context	Fault tolerance and availability issues due to service dependency
No static code sharing	Scalability and throughput issues due to service dependency
	Increased risk due to runtime changes

When to Use

The shared service technique is good to use in highly polyglot environments (those with multiple heterogeneous languages and platforms), and also when shared functionality tends to change often. While changes in a shared service tend to be much more agile overall than with the shared library technique, be careful of runtime side-effects and risks to services needing the shared functionality.

Sidecars and Service Mesh

Perhaps the most common response to any question posed by an architect is "It depends!" No issue in distributed architectures better illustrates this ambiguity better than operational coupling.

One of the design goals of microservices architectures is a high degree of decoupling, often manifested in the advice "Duplication is preferable to coupling." For example, let's say that two Sysops Squad services need to pass customer information, yet the domain-driven design bounded context insists that implementation details remain private to the service. Thus, a common solution allows each service its own internal representation of entities such as `Customer`, passing that information in loosely coupled ways such as name-value pairs in JSON. Notice that this allows each service to change its internal representation at

will, including the technology stack, without breaking the integration. Architects generally frown on duplicating code because it causes synchronization issues, semantic drift, and a host of other issues, but sometimes forces exist that are worse than the problems of duplication, and coupling in microservices often fits that bill. Thus, in microservices architecture, the answer to the question of "should we duplicate or couple to some capability?" is likely *duplicate*, whereas in another architecture style such as a service-based architecture, the correct answer is likely *couple*. It depends!

When designing microservices, architects have resigned themselves to the reality of implementation duplication to preserve decoupling. But what about the type of capabilities that *benefit* from high coupling? For example, consider common operational capabilities such as monitoring, logging, authentication and authorization, circuit breakers, and a host of other operational abilities that each service should have. But allowing each team to manage these dependencies often descends into chaos. For example, consider a company like Penultimate Electronics trying to standardize on a common monitoring solution to make it easier to operationalize the various services. Yet if each team is responsible for implementing monitoring for their service, how can the operations team be sure they did? Also, what about issues such as unified upgrades? If the monitoring tool needs to upgrade across the organization, how can teams coordinate that?

The common solution that has emerged in the microservices ecosystem over the last few years solves this problem in an elegant way, by using the Sidecar pattern. This pattern is based on a much earlier architecture pattern defined by Alistair Cockburn, known as the *hexagonal architecture*, illustrated in Figure 8-12.

In this Hexagonal pattern, what we would now call the domain logic resides in the center of the hexagon, which is surrounded by ports and adaptors to other parts of the ecosystem (in fact, this pattern is alternately known as the *Ports and Adaptors Pattern*). While predating microservices by a number of years, this pattern has similarities to modern microservices, with one significant difference: data fidelity. The hexagonal architecture treated the database as just another adaptor that can be plugged in, but one of the insights from DDD suggests that data schemas and transactionality should be inside the interior—like microservices.

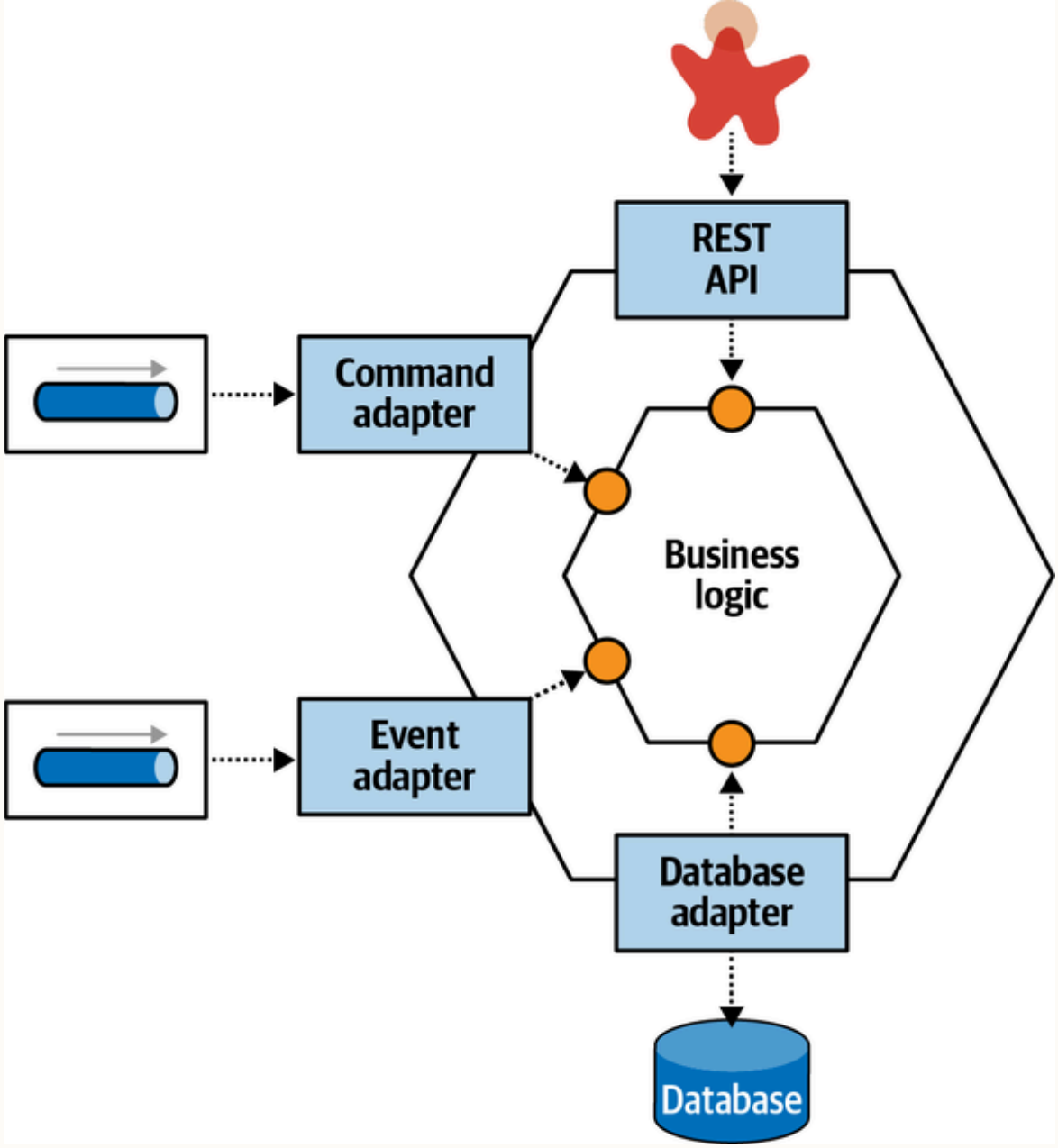


Figure 8-12. The Hexagonal pattern separated domain logic from technical coupling

The *Sidecar pattern* leverages the same concept as hexagonal architecture in that it decouples the domain logic from the technical (infrastructure) logic. For example, consider two microservices, as shown in Figure 8-13.

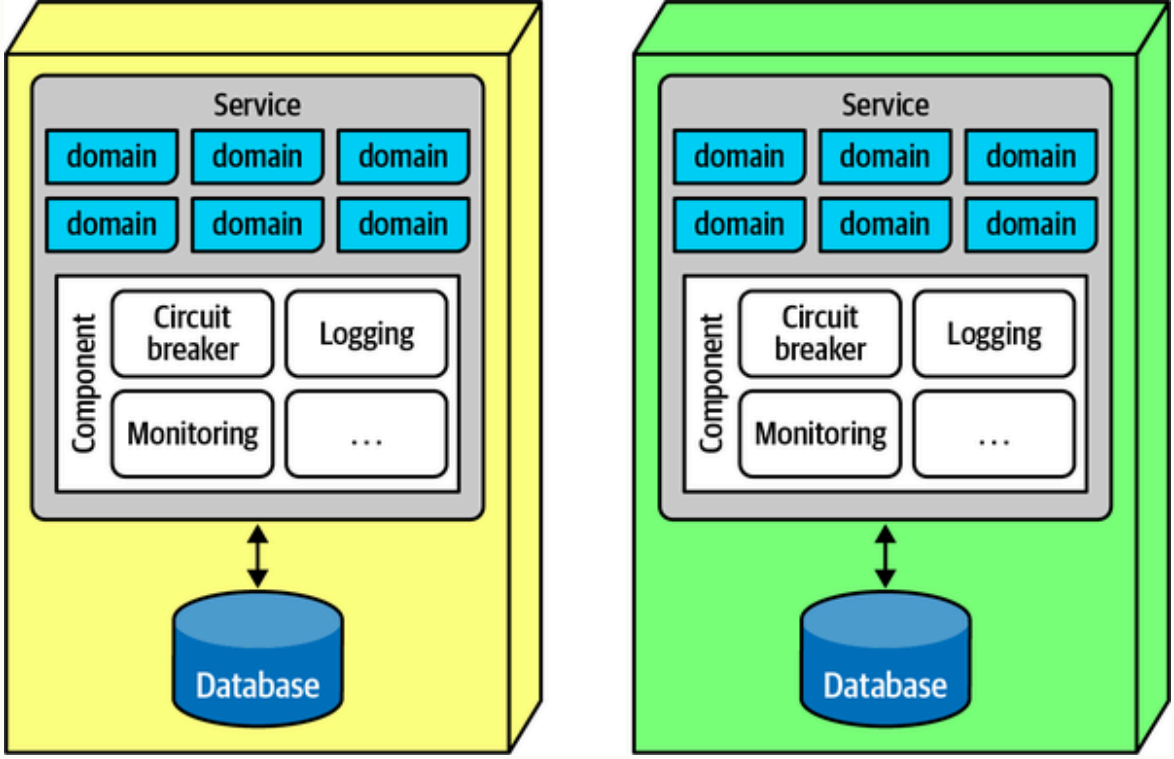


Figure 8-13. Two microservices that share the same operational capabilities

Here, each service includes a split between operational concerns (the larger components toward the bottom of the service) and domain concerns, pictured in the boxes toward the top of the service labeled "domain." If architects desire consistency in operational capabilities, the separable parts go into a sidecar component, metaphorically named for the sidecar that attaches to motorcycles, whose implementation is either a shared responsibility across teams or managed by a centralized infrastructure group. If architects can assume that every service includes the sidecar, it forms a consistent operational interface across services, typically attached via a service plane, shown in Figure 8-14.

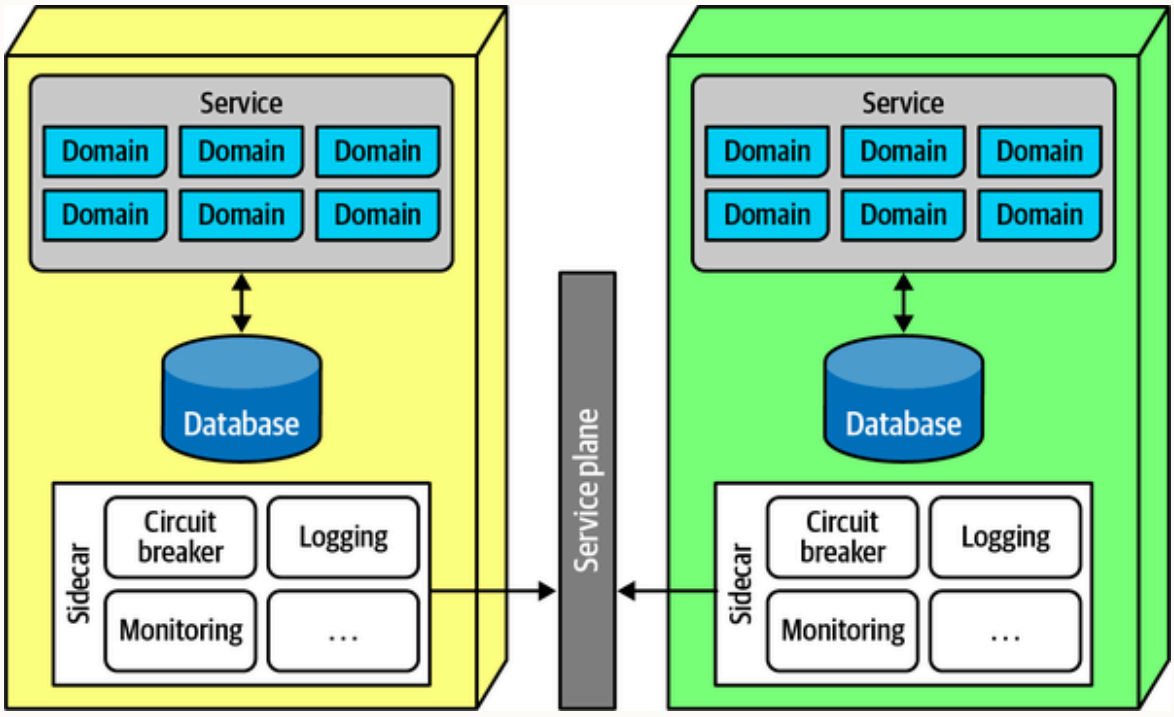


Figure 8-14. When each microservice includes a common component, architects can establish links between them for consistent control

If architects and operations can safely assume that every service includes the sidecar component (governed by fitness functions), it forms a service mesh, illustrated in Figure 8-15. The boxes to the right of each service all interconnect, forming a "mesh."

Having a mesh allows architects and DevOps to create dashboards, control operational characteristics such as scale, and a host of other capabilities.

The Sidecar pattern allows governance groups like enterprise architects a reasonable restraint over too many polyglot environments: one of the advantages of microservices is a reliance on integration rather than a common platform, allowing teams to choose the correct level of complexity and capabilities on a service-by-service basis. However, as the number of platforms proliferates, unified governance becomes more difficult. Therefore, teams often use the consistency of the service mesh as a driver to support infrastructure and other cross-cutting concerns across multiple heterogeneous platforms. For example, without a service mesh, if enterprise architects want to unify around a common monitoring solution, then teams must build a sidecar per platform that supports that solution.

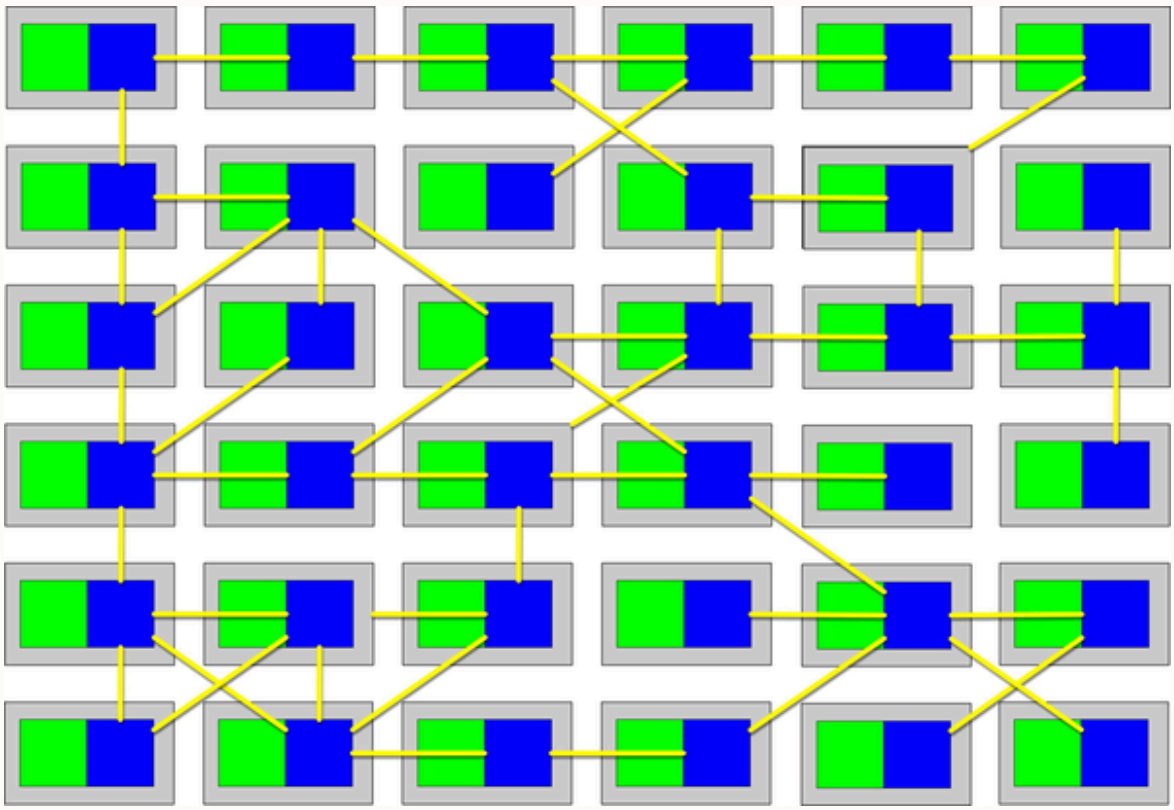


Figure 8-15. A service mesh is an operational link among services

The Sidecar pattern represents not only a way to decouple operational capabilities from domains—it's an *orthogonal reuse* pattern to address a specific kind of coupling (see "Orthogonal Coupling"). Often, architectural solutions require several types of coupling, such as our current example of domain versus operational coupling. An orthogonal reuse pattern presents a way to reuse some aspect counter to one or more seams in the architecture. For example, microservices architectures are organized around domains, but opera-

tional coupling requires cutting across those domains. A sidecar allows an architect to isolate those concerns in a cross-cutting, but consistent, layer through the architecture.

ORTHOGONAL COUPLING

In mathematics, two lines are *orthogonal* if they intersect at right angles, which also implies independence. In software architecture, two parts of an architecture may be *orthogonally coupled*: two distinct purposes that must still intersect to form a complete solution. The obvious example from this chapter is an operational concern such as monitoring, which is necessary but independent from domain behavior, like catalog checkout. Recognizing orthogonal coupling allows architects to find intersection points that cause the least entanglement between concerns.

While the Sidecar pattern offers a nice abstraction, it has trade-offs like all other architectural approaches, shown in Table 8-4.

TRADE-OFFS

Table 8-4. Trade-offs for the Sidecar pattern / service mesh technique

Advantages	Disadvantages
Offers a consistent way to create isolated coupling	Must implement a sidecar per platform
Allows consistent infrastructure coordination	Sidecar component may grow large/complex
Ownership per team, centralized, or some combination	

When to Use

The Sidecar pattern and service mesh offer a clean way to spread some sort of cross-cutting concern across a distributed architecture, and can be used by more than just operational coupling (see Chapter 14). It offers an architectural equivalent to the Decorator Design Pattern from the Gang of Four *Design Patterns* book (Addison Wesley)—it allows an architect to "decorate" behavior across a distributed architecture independent of the normal connectivity.

Sysops Squad Saga: Common Infrastructure Logic

Thursday, February 10, 10:34



Sydney peeped into Taylen's office on a foggy morning. "Hey, are you using the shared Message Dispatch library?"

Taylen replied, "Yes, we're trying to consolidate on that to get some consistency on message resolution."

Sydney said, "OK, but now we're getting double log messages—it looks like the library writes to the logs, but our service also writes to the log. Is that as it should be?"

"No," Taylen replied. "We definitely don't want duplicate log entries. That just makes everything confusing. We should ask Addison about that."

Consequently, Sydney and Taylen darkened Addison's door. "Hey, do you have a minute?"

Addison replied, "Always for you—what's up?"

Sydney said, "We've been consolidating a bunch of our duplicated code into shared libraries, and that's working well—we're getting better at identifying the parts that rarely change. But, now we've hit the problem that brings us here—who is supposed to be writing log messages? Libraries, services, or something else? And, how can we make that consistent?"

Addison said, "We've bumped into operational shared behavior. Logging is just one of them. What about monitoring, service discovery, circuit breakers, even some of our utility functions, like the `JSONtoXML` library that a few teams are sharing? We need a better way to handle this to prevent issues. That's why we're in the process of implementing a service mesh with this common behavior in a sidecar component."

Sydney said, "I've read about sidecars and service mesh—it's a way to share things across a bunch of microservices, right?"

Addison said, "Sort of, but not all kinds of *things*. The intent of the service mesh and sidecar is to consolidate operational coupling, not domain coupling. For example, just like in our case, we want consistency for logging and monitoring across all our services, but don't want each team to have to worry about that. If we consolidate logging code into the common sidecar that every service implements, we can enforce consistency."

Taylen asked, "Who owns the shared library? Shared responsibility across all the teams?"

Addison replied, "We thought about that, but we have enough teams now; we've built a shared infrastructure team that is going to manage and maintain the sidecar component. They have built the deployment pipeline to automatically test the sidecar once it's been bound into the service with a set of fitness functions."

Sydney said, "So if we need to share libraries between services, just ask them to put it in the sidecar?"

Addison said, "Be careful—the sidecar isn't meant to be used for just anything, only operational coupling."

"I'm not sure what that distinction is," Taylen said.

"Operational coupling includes the things we've been discussing—logging, monitoring, service discovery, authentication and authorization, and so on. Basically, it covers all the plumbing parts of the infrastructure that have no domain responsibility. But you should never put domain shared components, like the `Address` or `Customer` class, in the sidecar."

Sydney asked, "But why? What if I need the same class definition in two services? Won't putting it in the sidecar make it available to both?"

Addison replied, "Yes, but now you are increasing coupling in exactly the way we try to avoid in microservices. In most architectures, a single implementation of that service would be shared across the teams that need it. However, in microservices, that creates a coupling point, tying several services together in an undesirable way—if one team changes the shared code, every team must coordinate with that change. However, the architects could decide to put the shared library in the sidecar—it is, after all a technical capability. Neither answer is unambiguously correct, making this an architect decision and worthy of trade-off analysis. For example, if the `Address` class changes and both services rely on it, they must both change—the definition of coupling. We handle those issues with contracts. The other issue concerns size: we don't want the sidecar to become the biggest part of the architecture. For example, consider the `JSONtoXML` library we were discussing before. How many teams use that?"

Taylen said, "Well, any team that has to integrate with the mainframe system for anything—probably 5 out of, what, 16 or 17 teams?"

Addison said, "Perfect. OK, what's the trade-off of putting the `JSONtoXML` in the sidecar?"

Sydney answered, "Well, that means that every team automatically has the library and doesn't have to wire it in through dependencies."

"And the bad side?" asked Addison.

"Well, adding it to the sidecar makes it bigger, but not by much—it's a small library." said Sydney.

"That's the key trade-off for shared utility code—how many teams need it versus how much overhead does it add to every service, particularly ones that don't need it."

"And if less than one-half the teams use it, it's probably not worth the overhead," Sydney said.

"Right! So, for now, we'll leave that out of the sidecar and perhaps reassess in the future," said Addison.

> *ADR: Using a Sidecar for Operational Coupling*
>
> *Context*
> *Each service in our microservices architecture requires common and consistent operational behavior; leaving that responsibility to each team introduces inconsistencies and coordination issues.*
>
> *Decision*
> *We will use a sidecar component in conjunction with a service mesh to consolidate shared operational coupling.*
>
> *The shared infrastructure team will own and maintain the sidecar for service teams; service teams act as their customers. The following services will be provided by the sidecar:*
>
> - *Monitoring*
> - *Logging*
> - *Service discovery*
> - *Authentication*
> - *Authorization*
>
> *Consequences*
> *Teams should not add domain classes to the sidecar, which encourages inappropriate coupling.*
>
> *Teams work with the shared infrastructure team to place shared, operational libraries in the sidecar if enough teams require it.*

Code Reuse: When Does It Add Value?

Many architects fail to properly assess trade-offs when they encounter some situations, which isn't necessarily a deficiency—many trade-offs become obvious only after the fact.

Reuse is one of the most abused abstractions, because the general view in organizations is that *reuse* represents a laudable goal that teams should strive for. However, failing to evaluate all the trade-offs associated with reuse can lead to serious problems within architecture.

The danger of too much reuse was one of the lessons many architects learned from the early 21st century trend of orchestration-driven service-oriented architecture, where one of the primary goals for many organizations was to maximize reuse.

Consider the scenario from an insurance company, illustrated in Figure 8-16.

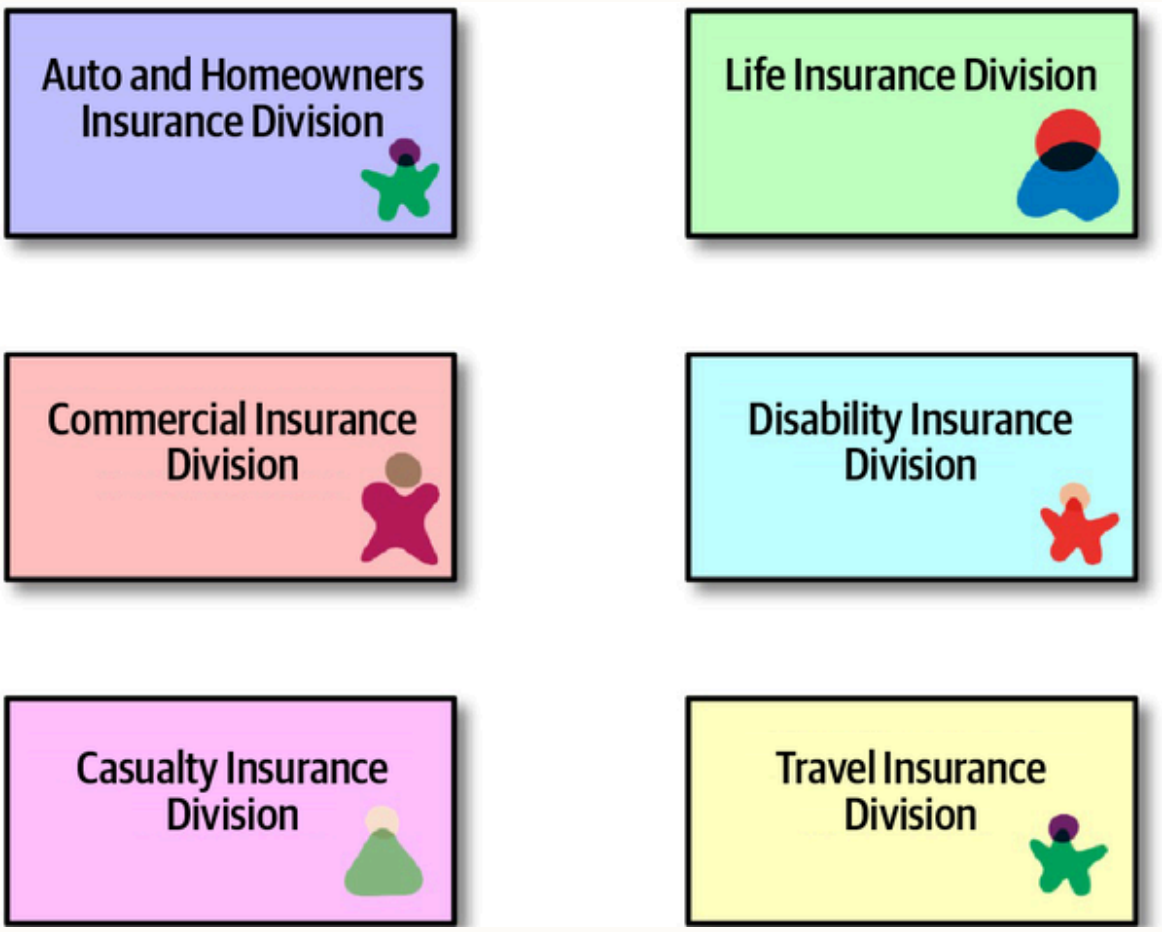


Figure 8-16. Each domain within a large insurance company has a view of the customer

Each division in the company has some aspect of customers it cares about. Years ago, architects were instructed to keep an eye out for this type of commonality; once discovered, the goal was to consolidate the organizational view of customer into a single service, shown in Figure 8-17.

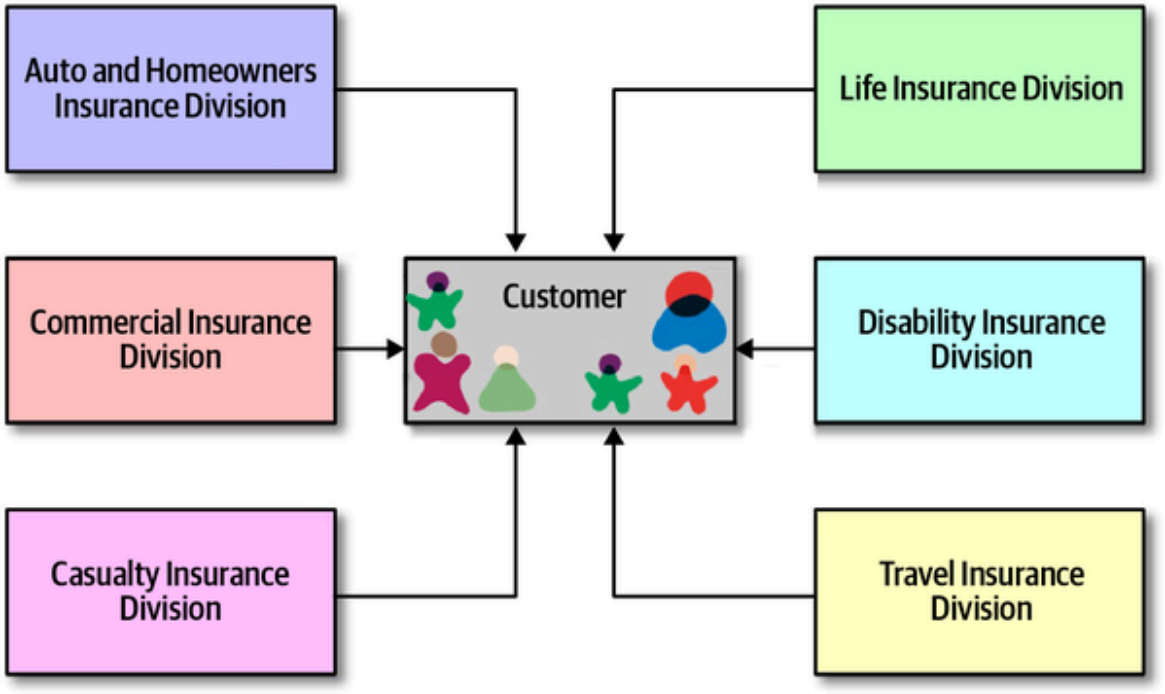


Figure 8-17. Unifying on a centralized Customer service

While the picture in Figure 8-17 may seem logical, it's an architectural disaster for two reasons. First, if all institutional information about a key entity like Customer must reside in a single place, that entity must be complex enough to handle any domain and scenario, making it difficult to use for simple things.

Secondly, though, it creates *brittleness* within the architecture. If every domain that needs customer information must get it from a single place, when that place changes, everything breaks. For example, in our example, what happens when `CustomerService` needs to add new capabilities on behalf of one of the domains? That change could potentially impact every other domain, requiring coordination and testing to ensure that the change hasn't "rippled" throughout the architecture.

What architects failed to realize is that reuse has two important aspects; they got the first one correct: abstraction. The way architects and developers discover candidates for reuse is via abstraction. However, the second consideration is the one that determines utility and value: *rate of change*.

Observing that some reuse causes brittleness begs the question about how that kind of reuse differs from the kinds we clearly benefit from. Consider things that everyone successfully reuses: operating systems, open source frameworks and libraries, and so on. What distinguishes those from assets that project teams build? The answer is *slow rate of change*. We benefit from technical coupling, like operating systems and external frameworks, because they have a well-understood rate of change and update cadence. Internal domain capabilities or quick-changing technical frameworks make terrible coupling targets.

TIP

Reuse is derived via abstraction but operationalized by slow rate of change.

Reuse via Platforms

Much press exists extolling the virtue of platforms within organizations, almost to the point of *semantic diffusion*. However, most agree that the *platform* is the new target of reuse within organizations, meaning that for each distinguishable domain capability, the organization builds a platform with a well-defined API to hide the implementation details.

Slow rate of change drives this reasoning. As we discuss in Chapter 13, an API can be designed to be quite loosely coupled to callers, allowing for an aggressive internal rate of change of implementation details without breaking the API. This, of course, doesn't protect the organization from changes to the semantics of the information it must pass between domains, but by careful design of encapsulation and contracts, architects can limit the amount of breaking change and brittleness in integration architecture.

Sysops Squad Saga: Shared Domain Functionality

Tuesday, February 8, 12:50



With Addison's approval, the development team had decided to split the core ticketing functionality into three separate services: a customer-facing Ticket Creation service, a Ticket Assignment service, and a Ticket Completion service. However, all three services used common database logic (queries and updates) and shared a set of database tables in the ticketing data domain.

Taylen wanted to create a shared data service that would contain the common database logic, thus forming a database abstraction layer, as shown in Figure 8-18.

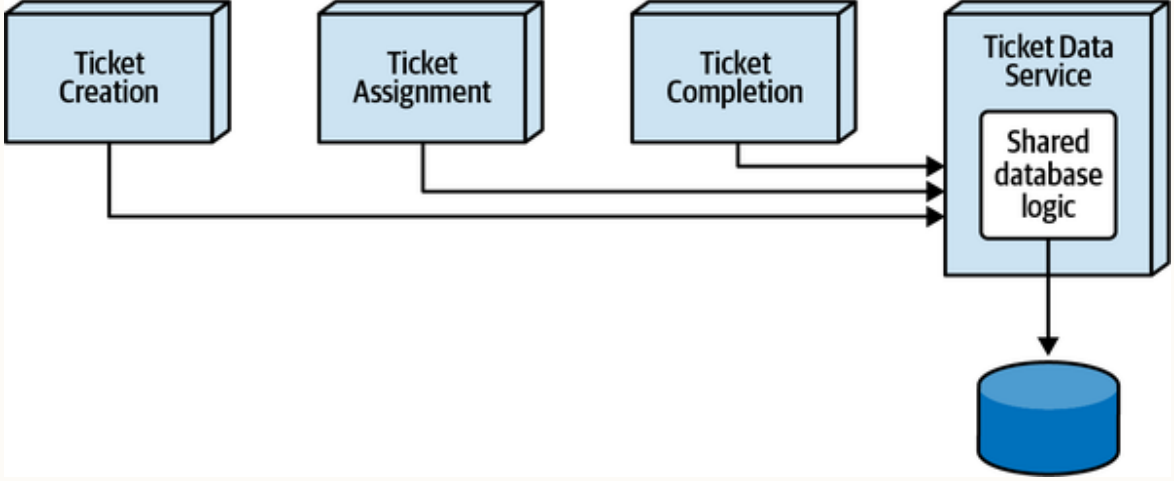


Figure 8-18. Option using a shared Ticket Data service for common database logic for the Sysops Squad ticketing services

Skyler hated the idea and wanted to use a single shared library (DLL) that each service would include as part of the build and deployment, as illustrated in Figure 8-19.

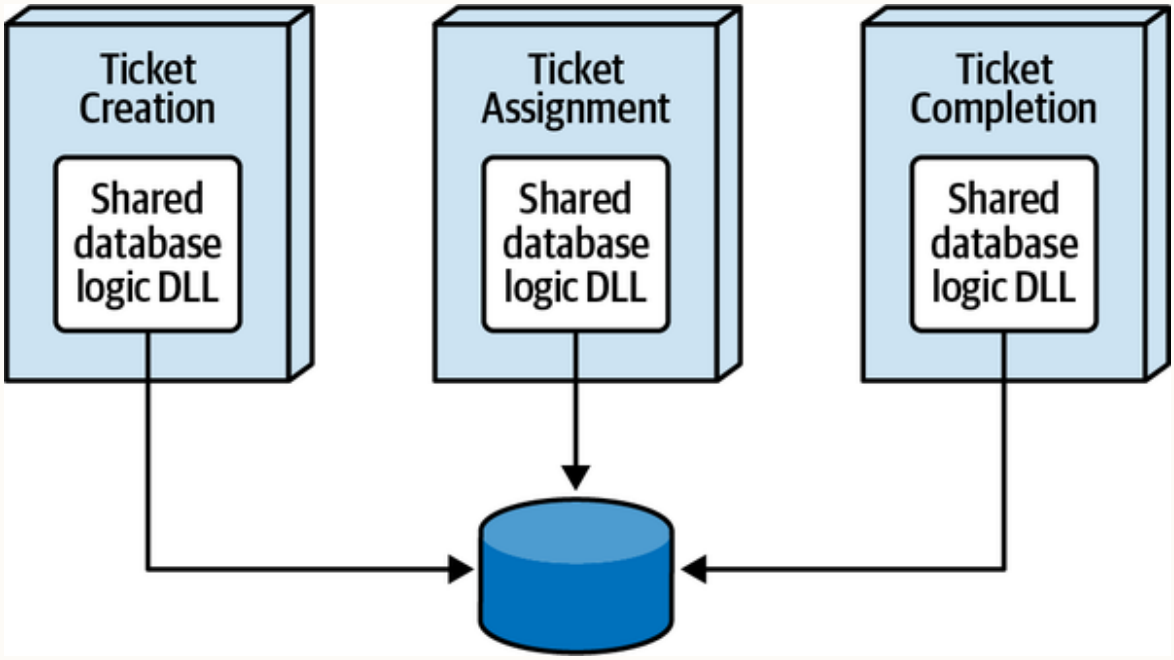


Figure 8-19. Option using a shared library for common database logic for the Sysops Squad ticketing services

Both developers met with Addison to resolve this roadblock.

"So, Addison, what is your opinion? Should the shared database logic be in a shared data service or a shared library?" asked Taylen.

"It's not about opinions," said Addison. "It's about analyzing the trade-offs to arrive at the most appropriate solution for the core shared ticketing database functionality. Let's do a hypothesis-based approach and hypothesize that the most appropriate solution is to use the shared data service."

"Hold on," said Skyler. "It's simply not a good architectural solution for this problem."

"Why?" asked Addison, prompting Skyler to start thinking in terms of trade-offs.

“First of all,” said Skyler, “all three services would need to make an interservice call to the shared data service for *every* database query or update. We’re going take a serious performance hit if we do that. Furthermore, if the shared data service goes down, all three of those services become nonoperational.”

“So?” said Taylen. “It's all backend functionality, so who cares? The backend functionality doesn't have to be that fast, and services come up fairly quickly if they fail.”

“Actually,” said Addison, “it's not all backend functionality. Don't forget, the Ticket Creation service is customer facing, and it would be using the same shared data service as the backend ticketing functionality.”

“Yeah, but *most* of the functionality is still backend,” said Taylen, with a little less confidence than before.

“So far,” said Addison, “it looks like the trade-off for using the shared data service is performance and fault tolerance for the ticketing services.”

“Let's also not forget that any changes made to the shared data service are runtime changes. In other words,” said Skyler, “if we make a change and deploy the shared data service, we could possibly break something.”

“That’s why we test,” said Taylen.

“Yeah, but if you want to reduce risk you would have to test all of the ticketing services for every change to the shared data service, which increases testing time significantly. With a shared DLL, we could version the shared library to provide backward compatibility,” said Skyler.

“OK, we will add increased risk for changes and increased testing effort to the trade-offs as well,” said Addison. “Also, let’s not forget that we would have extra coordination from a scalability standpoint. Every time we create more instances of the ticket creation service, we would have to make sure we create more instances of the shared data service as well.”

“Let's not keep focusing so much on the negatives.” said Taylen. “How about the positives of using a shared data service?”

“OK,” said Addison, “let's talk about the benefits of using a shared data service.”

“Data abstraction, of course,” said Taylen. “The services wouldn’t have to worry about any database logic. All they would have to do is make a remote service call to the shared data service.”

“Any other benefits?” asked Addison.

“Well,” said Taylen, “I was going to say centralized connection pooling, but we would need multiple instances anyway to support the customer ticket creation service. It would help, but it’s not a major game changer since there are only three services without a lot of instances of each service. However, change control would be so much easier with a shared data service. We wouldn’t have to redeploy any of the ticketing services for database logic changes.”

“Let’s take a look at those shared class files in the repository and see historically how much change there really is for that code,” said Addison.

Addison, Taylen, and Skyler all looked at the repository history for the shared data logic class files.

“Hmm...” said Taylen, “I thought there were a lot more changes to that code than what is showing up in the repo. OK, so I guess the changes are fairly minimal for the shared database logic after all.”

Through the conversation of discussing trade-offs, Taylen started to realize that the negatives of a shared service seemed to outweigh the positives, and there was no real compelling justification for putting the shared database logic in a shared service. Taylen agreed to put the shared database logic in a shared DLL, and Addison wrote an ADR for this architecture decision:

ADR: Use of a Shared Library for Common Ticketing Database Logic

Context
The ticketing functionality is broken into three services: Ticket Creation, Ticket Assignment, and Ticket Completion. All three services use common code for the bulk of the database queries and update statements. The two options are to use a shared library or create a shared data service.

Decision
We will use a shared library for the common ticketing database logic.

Using a shared library will improve performance, scalability, and fault tolerance of the customer-facing Ticket Creation service, as well as for the Ticket Assignment service.

We found that the common database logic code does not change much and is therefore fairly stable code. Furthermore, change is less risky for the common database logic because services would need to be tested and redeployed. If changes are needed, we will apply versioning where appropriate so that not all services need to be redeployed when the common database logic changes.

Using a shared library reduces service coupling and eliminates additional service dependencies, HTTP traffic, and overall bandwidth.

Consequences
Changes to the common database logic in the shared DLL will require the ticketing services to be tested and deployed, therefore reducing overall agility for common database logic for the ticketing functionality.

Service instances will need to manage their own database connection pool.

Previous chapter

‹ II. Putting Things Back Together

Next chapter

9. Data Ownership and Distributed Transactions ›